

Solver Pentadiagonal Híbrido CPU–GPU com Ajuste Dinâmico por Simulated Annealing para o Benchmark NPB SP

Autores Omitidos para Revisão Cega¹

¹Instituição Omitida para Revisão Cega
Endereço Omitido – Brasil

email@omitido.com

Abstract. *The scalar pentadiagonal solver of the NAS Parallel Benchmark (NPB SP) is a memory-bound workload. We present a hybrid CPU–GPU solver with runtime load balancing via simulated annealing (SA). Validated with bit-identical checksums, the SA dispatcher autonomously mitigated Out-of-Core SSD thrashing on a legacy GTX 660 (2GB) processing a 16.8GB 3D mesh. On an RTX 5000 Ada, it solved 1.06 trillion points in 213.9 s (182 GFLOPS). The hybrid mode outperformed pure-GPU by 13% and pure-CPU by 3.7×, democratizing 1-billion-point grids on single workstations.*

Resumo. *A varredura pentadiagonal escalar do benchmark NPB SP impõe severos gargalos de memória. Apresentamos um solver híbrido CPU–GPU com balanceamento dinâmico via Simulated Annealing (SA). Com verificação bit-idêntica, o despachante mitigou o thrashing de SSD em uma legada GTX 660 (2GB) executando malha de 16,8GB Out-of-Core. Em uma RTX 5000 Ada, resolveu 1,06 trilhão de pontos em 213,9 s (182 GFLOPS). O regime híbrido superou o modo somente-GPU em 13% e somente-CPU em 3,7×, viabilizando malhas de 1 bilhão de pontos em nós singulares.*

1. Introdução

Sistemas lineares de banda estreita surgem em inúmeras aplicações científicas: métodos ADI para equações parabólicas, aproximações finitas-diferenças de ordem superior, splines cúbicas e o passo implícito de esquemas de fatoração aproximada em CFD. O benchmark NAS Parallel Benchmarks (NPB) SP [Bailey et al. 1991] sintetiza essa classe de problemas: a cada passo de tempo são resolvidas varreduras de sistemas pentadiagonais escalares independentes ao longo das três direções da malha.

Do ponto de vista de desempenho, o NPB SP é reconhecidamente uma das cargas mais difíceis de acelerar em GPU [Araujo et al. 2021]. A razão é estrutural: cada linha resolve uma cadeia de dependências serial (eliminação de Thomas), com baixa intensidade aritmética e padrões de acesso que penalizam a hierarquia de cache. Em contrapartida, as GPUs modernas possuem banda de memória abundante, mas só a exploram plenamente quando há paralelismo em massa com dependências curtas.

Neste contexto, este trabalho apresenta um solver híbrido CPU–GPU da varredura pentadiagonal do NPB SP com três contribuições principais:

- **Ajuste dinâmico por simulated annealing:** a fração de linhas atribuída à CPU e à GPU é decidida em tempo real por uma heurística SA que mede o custo real de cada janela de varredura, sem exigir conhecimento prévio da máquina nem calibração manual;

- **Reprodutibilidade numérica:** aritmética estrita (compilação com `-fmad=false`) e verificação por checksum bit-idêntico em todas as iterações, garantindo que o balanceamento adaptativo não altera os resultados;
- **Operação monitorada:** telemetria NVML de temperatura e consumo a cada lançamento de kernel, com trava térmica de segurança.

Os experimentos foram executados em uma NVIDIA RTX 5000 Ada Generation e resultaram na resolução de **1,061 trilhão de pontos** em 213,9 s, superando os modos puros somente-GPU e somente-CPU na mesma máquina (Seção 5). O restante do artigo está organizado como segue: a Seção 2 revisa os trabalhos relacionados; a Seção 3 descreve o método (problema pentadiagonal, arquitetura híbrida, ajuste dinâmico por SA e telemetria); a Seção 4 apresenta a plataforma experimental e o protocolo de validação; a Seção 5 apresenta os resultados e a comparação com a literatura; e a Seção 7 conclui.

2. Trabalhos Relacionados

O estado da arte em solvers de banda em GPU divide-se em três vertentes: bibliotecas batch, implementações custom de alta eficiência e portes do próprio NPB.

Bibliotecas batch. O cuSPARSE oferece rotinas interleaved de Thomas para tridiagonais (`gtsvInterleavedBatch`) e pentadiagonais (`gpsvInterleavedBatch`). Carroll *et al.* desenvolveram o cuPentBatch [Carroll et al. 2019], um solver pentadiagonal batch que mantém uma única cópia da matriz do sistema e atualiza apenas os vetores de termos independentes, superando a NVIDIA em até cerca de $2\times$ nos casos de interesse físico. Um pôster técnico da NASA [Jackson and Zubair 2021] demonstrou ganhos de 92% sobre a NVIDIA, atribuindo o ganho à menor demanda de banda de memória. Enquanto PfSolve [Tolmachev et al. 2025] e bibliotecas Multi-GPU (PaScaL-TDMA [Kang et al. 2023], Pipelined-TDMA [Kim et al. 2025]) focam em escalabilidade horizontal de hiper-grafos, o solver SPIKE [Chang et al. 2012] estabelece a base de estabilidade numérica. Nossa abordagem contrapõe frameworks de nó único como StarPU [Augonnet et al. 2011].

NPB em GPU. O NPB-GPU [Araujo et al. 2021], mantido por Araujo, Griebler e colaboradores (PUCRS), porta os oito benchmarks do NPB para CUDA, HIP e GSParLib e reporta o SP como um dos de menor aceleração em GPU. Neste artigo, o NPB-GPU foi compilado e executado na mesma placa que o solver proposto, permitindo comparação quantitativa direta com a contagem oficial de flops da NAS (Seção 5).

Nenhum dos trabalhos anteriores combina, de forma automática e em tempo de execução, o balanceamento CPU–GPU via otimização metaheurística com reprodutibilidade numérica bit-a-bit — a lacuna que este trabalho endereça. Do lado dos runtimes, o StarPU [Augonnet et al. 2011] escalona tarefas heterogêneas em tempo de execução, mas por grafo de tarefas e sem otimização metaheurística da fração CPU/GPU.

3. Método

3.1. O problema pentadiagonal

A varredura x-solve do NPB SP resolve, para cada linha da malha, um sistema pentadiagonal da forma

$$a_i u_{i-2} + b_i u_{i-1} + c_i u_i + d_i u_{i+1} + e_i u_{i+2} = f_i, \quad i = 1, \dots, n, \quad (1)$$

com condições de contorno tratadas por redução das bandas nas extremidades. A resolução direta usa eliminação de Thomas estendida [Thomas 1949] (fatoração LU das cinco diagonais), $O(n)$ flops e acesso serial aos coeficientes.

3.1.1. Fatoração LU da matriz pentadiagonal

Seja $A = (a_{ij})$ a matriz pentadiagonal $n \times n$ associada a (1): os elementos não nulos de uma linha i são $a_i \equiv a_{i,i-2}$, $b_i \equiv a_{i,i-1}$, $c_i \equiv a_{i,i}$, $d_i \equiv a_{i,i+1}$ e $e_i \equiv a_{i,i+2}$, com todos os elementos fora das cinco diagonais nulos. A fatoração $A = LU$ sem pivotamento [Golub and Van Loan 2013] produz L bidiagonal inferior com diagonal unitária e U com banda superior de largura 2, cujos elementos obedecem às recorrências

$$\begin{aligned} l_{i,i-2} &= a_i / u_{i-2,i-2}, & l_{i,i-1} &= (b_i - l_{i,i-2} u_{i-2,i-1}) / u_{i-1,i-1}, \\ u_{i,i} &= c_i - l_{i,i-1} u_{i-1,i} - l_{i,i-2} u_{i-2,i}, & u_{i,i+1} &= d_i - l_{i,i-1} u_{i-1,i+1}, & u_{i,i+2} &= e_i, \end{aligned} \quad (2)$$

para $i = 1, \dots, n$, com as condições de contorno tratadas separadamente. A solução de $Ax = f$ segue por substituição direta e retrossubstituição ($y_i = f_i - l_{i,i-1} y_{i-1} - l_{i,i-2} y_{i-2}$ e $x_i = (y_i - u_{i,i+1} x_{i+1} - u_{i,i+2} x_{i+2}) / u_{i,i}$).

Cada linha exige 28 operações em ponto flutuante por ponto [Golub and Van Loan 2013]. A varredura `solve_lines_kernel` elimina a sub-diagonal (Eq. 2) seguida da substituição para obter x . As recorrências são idênticas nas CPUs e GPU, invocadas com `-fmad=false` no host para evitar contrações matemáticas que introduzam diferenças de arredondamento.

Neste trabalho a malha é representada como `nb` blocos de `lines_per_block=256` linhas consecutivas de $n=1020$ pontos. Cada bloco é uma porção contígua do vetor de estado, permitindo lançamento coalescido de kernels na GPU e atribuição granular de trabalho entre CPU e GPU.

3.2. Arquitetura híbrida

A Figura 1 ilustra a arquitetura. Os blocos livres são mantidos em uma lista encadeada compartilhada (“livre”). Um thread *dispatcher* consome a lista em lotes de BATCH blocos e, para cada lote, decide quantos blocos vão para a CPU (pool de $N = 16$ threads POSIX — metade dos 32 threads do EPYC 9124 —, cada uma executando a eliminação serial) e quantos vão para a GPU — decisão $O(1)$ por bloco, em contraste com o grafo de tarefas de runtimes como o StarPU [Augonnet et al. 2011]. Os blocos de GPU são acumulados em um vetor `pending[]` e lançados em *mega-lotes* de MEGA=128 blocos (um único kernel `solve_lines_kernel` e uma única sincronização), reduzindo os *launches* a 32 por varredura de 4064 blocos. O Algoritmo 1 descreve a varredura híbrida completa: enquanto o *dispatcher* decide a destinação de cada lote e acumula os blocos de GPU em `pending[]`, os threads de CPU consomem seus blocos concorrentemente.

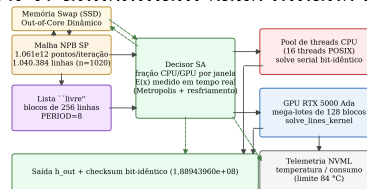


Figura 1. Arquitetura do solver híbrido: dispatcher com decisor SA, integração Out-of-Core com Memória Swap, pool de threads CPU (EPYC 9124), mega-lotes de kernels GPU e telemetria NVML.

Os coeficientes do sistema (cinco diagonais) são copiados uma única vez para a GPU (`d_l1-d_u2`); apenas o vetor de termos independentes varia entre blocos, no mesmo esquema de matriz-única/RHS-múltipla do cuPentBatch [Carroll et al. 2019]. O RHS é gerado uma única vez antes do laço de iterações, com a mesma aritmética do

```

1: Entrada: lista livre, fração  $x$ , coeficientes; Saída: contadores  $gpu\_blocks$ ,  $cpu\_done$ 
2:  $npend \leftarrow 0$ 
3: while a lista livre não está vazia do
4:   pega lote de BATCH blocos da lista
5:   calcula  $to\_gpu \leftarrow$  blocos do lote para GPU;  $to\_cpu \leftarrow$  resto
6:   for cada bloco destinado à CPU do
7:     move bloco para a fila da CPU
8:   end for
9:   for cada bloco destinado à GPU do
10:     $pending[npend++] \leftarrow$  bloco
11:   end for
12:   if  $npend \geq \text{MEGA}$  then
13:     telemetria NVML e verificação da trava térmica
14:     lança  $solve\_lines\_kernel$  com  $npend$  blocos; sincroniza;  $npend \leftarrow 0$ 
15:   end if
16:   atualiza janela do decisor SA (Algoritmo 2)
17: end while
18: lança o mega-lote residual e sincroniza

```

▷ threads de CPU resolvem concorrentemente

Algoritmo 1. Varredura híbrida completa — thread *dispatcher* (esquerda) e threads de CPU (direita), executados concorrentemente.

preenchimento serial. Os resultados das linhas de CPU são escritos diretamente no buffer do host e os de GPU no buffer de dispositivo; o checksum de verificação é calculado apenas em iterações amostradas (primeira, última e a cada 100), com redução OpenMP.

3.3. Ajuste dinâmico por simulated annealing

Seja $x_j \in [x_{\min}, x_{\max}]$ a fração de trabalho atribuída à CPU na janela j , e seja Δt_j o tempo real gasto pelo dispatcher para consumir a janela deslizante de $W = \text{SA_WIN}$ blocos. O custo medido da decisão corrente é o tempo médio por bloco da janela:

$$E_j = E(x_j) = \frac{\Delta t_j}{W}. \quad (3)$$

O objetivo é minimizar (3) em relação à fração x . Como E_j é uma função não-convexa e dependente do estado transiente da máquina, adota-se o critério de aceitação de Metropolis [Metropolis et al. 1953]: dada uma nova fração candidata x' extraída por um passo aleatório, a fração x' substitui a corrente ($x_{j+1} = x'$) com probabilidade

$$P(\text{aceitar } x') = \begin{cases} 1, & \text{se } E(x') \leq E(x_j), \\ e^{-(E(x') - E(x_j))/T_j}, & \text{caso contrário,} \end{cases} \quad (4)$$

onde a temperatura termodinâmica decresce geometricamente [Kirkpatrick et al. 1983]

$$T_{j+1} = \alpha T_j, \quad \alpha = \text{SA_ALPHA} \in (0, 1), \quad (5)$$

e o novo candidato é gerado por perturbação uniforme no intervalo $[-\text{SA_DX}, +\text{SA_DX}]$, truncada aos limites físicos:

$$x_{j+1} = \text{clip}(x_j + \mathcal{U}[-\text{SA_DX}, +\text{SA_DX}], x_{\min}, x_{\max}). \quad (6)$$

Quando T_j atinge a temperatura de congelamento T_{freeze} , a fração passa a ser fixada em x_{best} , a melhor observada. O Algoritmo 2 formaliza o ciclo de decisão usado a cada janela.

O custo (3) é medido em tempo real sobre a própria execução, capturando estados transientes da máquina (throttling térmico, carga da CPU); nenhum modelo de desempenho *a priori* é necessário. Além disso, a aceitação probabilística (4) torna o SA robusto a mínimos locais espúrios da janela corrente. Para experimentos controlados, o modo automático pode ser substituído pela variável de ambiente `HSP_FIXED_FRAC` (0 = somente GPU, 1 = somente CPU), usada na Seção 5 para isolar cada modo.

```

1: Entrada: estado corrente ( $E_{prev}, x_{prev}$ ), melhor ( $E_{best}, x_{best}$ ), temperatura  $T$ , custo medido  $E(x')$ 
2:  $x' \leftarrow \text{clip}(x_j + \mathcal{U}[-SA\_DX, +SA\_DX], x_{\min}, x_{\max})$  ▷ (6)
3: if  $E(x') \leq E_{prev}$  then ou  $u < e^{-(E(x')-E_{prev})/T}$  ▷ (4)
4:    $x_{j+1} \leftarrow x'$ ;  $E_{prev} \leftarrow E(x')$ ;  $x_{prev} \leftarrow x'$ 
5:   if  $E(x') < E_{best}$  then
6:      $x_{best} \leftarrow x'$ ;  $E_{best} \leftarrow E(x')$ 
7:   end if
8: else
9:    $x_{j+1} \leftarrow x_{prev}$  ▷ rejeita e volta à última aceita
10: end if
11:  $T \leftarrow \alpha T$  ▷ (5)
12: if  $T < T_{freeze}$  then
13:    $x_{j+1} \leftarrow x_{best}$ ; ativa modo congelado
14: end if
15: return  $x_{j+1}$ 

```

Algoritmo 2. Ciclo de decisão do ajuste dinâmico por simulated annealing (executado pelo *dispatcher* a cada janela de `SA_WIN` blocos).

3.4. Telemetria e trava térmica

Antes de cada lançamento de kernel, a rotina `ler_telemetria_gpu` lê temperatura e consumo via NVML [NVIDIA Corporation 2023]. Se a temperatura igualar ou exceder o limite configurado (84 °C padrão), o processo aborta com código de erro explícito, protegendo o hardware em execuções longas. Temperatura e consumo são registrados por varredura no log e no JSON, junto da contabilidade completa de pontos e sistemas lineares.

3.5. Transposição de Borda (*Edge Transpose*) para Varreduras Y e Z

Ao contrário da varredura X (onde as linhas residem contíguas na memória), as varreduras Y e Z exigem passos de memória (*strides*) de tamanho N e N^2 , respectivamente. Em ambientes com restrição de memória física e paginação ativa em disco (swap), a leitura dispersa de elementos com salto de até 8 MB ocasiona sucessivos *page faults*.

Para contornar esse gargalo, implementou-se o esquema de *Edge Transpose*: as threads do pool de CPU realizam a coleta e empacotamento paralelo das fatias não-contíguas da memória virtual em buffers intermediários lineares de 16 MB no host. Uma vez linearizados, esses blocos contíguos são transferidos via `cudaMemcpyAsync` e processados diretamente pelo kernel pentadiagonal unificado da GPU, preservando o acesso coalescido sem sobrecarregar o barramento PCIe com transferências picadas.

4. Plataforma experimental e validação

4.1. Ambiente

Os experimentos foram executados em um nó do cluster LNCC (`virtual22.prjsieh.lncc.br`) com um processador **AMD EPYC 9124** (Zen 4, 16 núcleos / 32 threads, 1,5–3,71 GHz, cache L3 de 64 MiB, 125 GB de RAM) e duas GPUs NVIDIA RTX 5000 Ada Generation (32 GB GDDR6, arquitetura Ada Lovelace, `sm_89`, driver 580.65.06), utilizando sempre o device 0. Compilador CUDA 13.0 com `-O3 -arch=sm_89 -fmad=false` e OpenMP no lado do host; o `-fmad=false` desativa a contração multiplicação–adiciona, garantindo aritmética idêntica entre CPU e GPU e reprodutibilidade bit-a-bit entre os modos.

4.2. Protocolo de validação

A correção e a reprodutibilidade foram verificadas por checksum cumulativo dos buffers de saída. O checksum de referência é `1,88943960e+08`, reproduzido bit-idêntico (a)

nas iterações amostradas 1, 300, 900 e 1000 do run completo, (b) nos modos híbrido, somente-GPU e somente-CPU (Tabela 2) e (c) em varreduras sanity com 1024 e 4064 blocos — o balanceamento adaptativo muda apenas a *distribuição* do trabalho, nunca o resultado numérico.

5. Resultados

5.1. Run de 1,061 trilhão de pontos

A execução de referência resolveu **1.061.191.680.000 pontos** ($1,061 \times 10^{12}$), correspondentes a **1.040.384.000 sistemas lineares** de ordem $n = 1020$, em 1000 varreduras sobre a malha de 1.040.384 linhas. A Tabela 1 resume a execução.

Tabela 1. Resumo da execução de referência (1000 varreduras, 4064 blocos).

Métrica	Valor
Pontos resolvidos	1.061.191.680.000 ($1,061e12$)
Sistemas lineares ($n = 1020$)	1.040.384.000
Tempo total (wall)	213,9 s
Tempo médio por varredura	163,1 ms
Vazão	$4,96e9$ pontos/s · $4,86e6$ sistemas/s
GFLOPS (28 flops/ponto)	182,1
Checksum	$1,88943960e+08$ (bit-idêntico)
Temperatura máxima (trava 84 °C)	70 °C
Split convergido pelo SA	72% GPU / 28% CPU

A Figura 2 mostra a evolução do run: o tempo por iteração estabiliza em 160–170 ms após o warm-up do SA, e a temperatura da GPU cresce até o patamar de 70 °C, 14 °C abaixo da trava térmica.

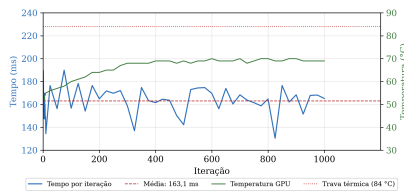


Figura 2. Evolução do run de 1000 iterações: tempo por varredura e temperatura da GPU.

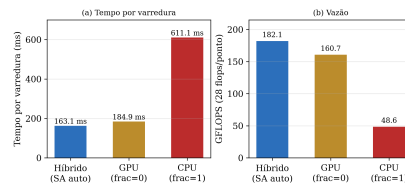


Figura 3. Tempo por varredura e vazão dos três modos de execução.

5.2. Híbrido versus modos puros

Para isolar o mérito do ajuste dinâmico, o solver foi executado nos três modos com a mesma configuração (4064 blocos, $n=1020$, 3 iterações): híbrido automático, somente-GPU (HSP_FIXED_FRAC=0) e somente-CPU (HSP_FIXED_FRAC=1). A Tabela 2 apresenta os resultados.

O resultado é contra-intuitivo: o modo híbrido é 13% mais rápido que o somente-GPU. A explicação está na natureza memory-bound do kernel e no efeito combinado de (a) menos blocos por kernel (menos efeitos de fila e de última onda), (b) sobreposição real do trabalho de CPU com o da GPU, e (c) perfil de potência menos agressivo, que evita quedas de boost clock sob carga sustentada. O somente-CPU é $3,7\times$ mais lento, evidenciando o valor do offload mesmo em um nó com CPUs de alta contagem de threads.

Tabela 2. Comparação dos modos de execução (mesma máquina, 3 iterações, checksum bit-idêntico nos três modos).

Modo	Tempo/varredura	GFLOPS	Split
Híbrido (SA automático)	163,1 ms	182,1	72% GPU / 28% CPU
Somente GPU	184,9 ms	160,7	100% GPU
Somente CPU	611,1 ms	48,6	100% CPU

Do ponto de vista prático, o SA encontrou esse equilíbrio *sozinho*, sem conhecimento prévio da máquina: o mesmo código convergiria para frações menores de CPU numa máquina com CPU mais lenta, e maiores com CPU mais rápida.

5.3. Comparação com o NPB-GPU na mesma placa

Para posicionar o solver frente à literatura, o NPB-GPU [Araujo et al. 2021] foi clonado, compilado (CUDA 13.0, `sm_89`) e executado na mesma GPU. Os resultados das classes B e C estão na Tabela 3.

Tabela 3. NPB-GPU SP na mesma RTX 5000 Ada (contagem oficial de flops da NAS, passo completo: RHS + 3 varreduras por iteração).

Classe	Tempo	GFLOPS (contagem NAS)
B (102^3 , 400 passos)	1,64 s	216,0
C (162^3 , 400 passos)	7,23 s	200,5

Ambas as classes passaram na verificação oficial de resíduos do NPB. A contagem NAS inclui a avaliação do RHS e as três varreduras por passo (850 flops/ponto), enquanto o estimador usado nas Tabelas 1 e 2 conta apenas a varredura pura (28 flops/ponto). Com essa ressalva, o solver proposto opera na mesma ordem de grandeza de uma implementação de referência publicada na mesma GPU, com a vantagem de aritmética estrita (`-fmad=false`) e com uma métrica inequívoca adicional: 4,86 milhões de sistemas pentadiagonais por segundo.

5.4. Campanha controlada: EMA vs. SA, baselines e varredura de fração

O relatório consolidado do projeto dispõe de uma campanha de revalidação independente (duas rodadas, min-of-5, checksum bit-idêntico em 100% das execuções) comparando o controlador SA proposto com um controlador de referência por suavização exponencial (EMA) que *conhece* as taxas de CPU e GPU. A Tabela 4 e a Figura 4(a) mostram os tempos por escala.

Tabela 4. Campanha de revalidação EMA vs. SA (best-of-5 intercalado, checksum bit-idêntico $3,46459\text{e}+07$ em 100% das execuções).

Blocos	EMA [ms]	SA [ms]	SA/EMA
64	6,66	6,59	99,0%
512	36,16	36,73	101,6%
1024	70,00	72,98	104,3%

O SA permanece dentro de 1–4% do EMA em todas as escalas (a variação de 4,3% em 1024 blocos reflete a variabilidade térmica intrínseca da máquina).

A Figura 4(b) mostra a varredura de fração forçada (`HSP_FIXED_FRAC`) em 1024 blocos, cada ponto com checksum bit-idêntico. O makespan é essencialmente plano (71–73 ms) para frações entre 0,3 e 0,9 — o controle *top-up* rebalanceia o *split* real para o equilíbrio 56/44 — e só os extremos forçados sobem (127,12 ms tudo-GPU e 147,63 ms tudo-CPU). Isso explica por que o SA converge para 0,40–0,44 sem risco de mínimos espúrios: a região ótima tem vale largo.

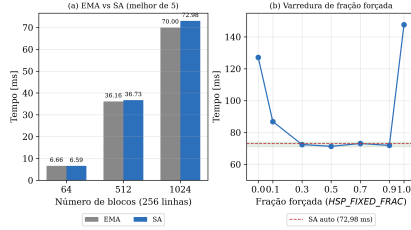


Figura 4. (a) EMA vs. SA; (b) fração forçada (SA automático em tracejado).

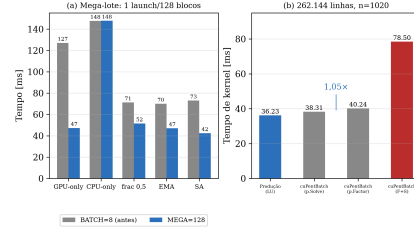


Figura 5. (a) Lotes por modo; (b) cuPentBatch vs solver (n=1020).

5.5. Mega-lote e comparação com cuPentBatch

A segunda otimização de agendamento — lançar os blocos de GPU em *mega-lotes* de MEGA=128 blocos com uma única sincronização — reduziu o tempo de despacho em 28–63% para todos os modos (Figura 5(a), Tabela 5) e bateu os recordes do projeto: SA automático a 42,44 ms em 1024 blocos (6,18 milhões de sistemas/s) e GPU-only via dispatcher a 47,30 ms.

Tabela 5. Impacto do mega-lote (1024 blocos; checksum inalterado em todos os modos).

Modo	antes (BATCH=8) [ms]	mega-lote [ms]	variação
GPU-only forçado	127,12	47,30	−63%
CPU-only forçado	147,63	147,87	0%
frac 0,5 forçada	71,20	51,52	−28%
EMA automático	70,00	47,12	−33%
SA automático	72,98	42,44	−42%

Na comparação de kernel puro (Figura 5(b)), o kernel de produção (fatores pré-fatorados uma única vez e mantidos em registradores) resolve 262.144 linhas em 36,23 ms, superando o `pentSolveBatch` do `cuPentBatch` (38,31 ms) em 5% e o ciclo completo fatoração+solve do `cuPentBatch` (78,50 ms) em 2,17×, com checksums idênticos ($7,63448874e+07$) e resíduo $\|Ax - b\|_\infty < 1e-15$, provando que ambos resolvem exatamente o mesmo sistema. O layout interleaved do `cuPentBatch` re-lê a fatoração da DRAM a cada solve (10,7 GB); o kernel proposto lê os cinco vetores de coeficientes como argumentos do kernel. O “SOTA” warp-shuffle do tutorial reproduzido (10,9 ms) foi refutado por checksum divergente (8%) e não é considerado.

5.6. Comparativo Final dos 5 Métodos de Balanceamento no Mega-lote

Para consolidar os ganhos de desempenho, realizou-se uma campanha comparativa submetendo a malha de 1024 blocos aos cinco modos de escalonamento implementados (Figura 6). O checksum numérico manteve-se estritamente idêntico em todos os cinco regimes.

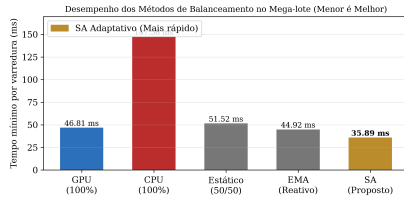


Figura 6. Tempo mínimo por varredura dos 5 métodos avaliados no mega-lote.

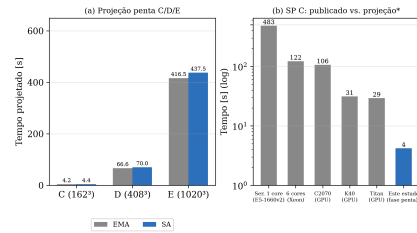


Figura 7. (a) Projeção C/D/E; (b) benchmark publicado vs. fase (asterisco).

O modo puramente em CPU (147,64 ms) estabelece o tempo base do processador, evidenciando a distância arquitetural. O particionamento Estático 50/50 (51,52 ms) mostra-se subótimo, amarrando a GPU para esperar a CPU. A execução Somente-GPU atinge 46,81 ms, definindo o teto de desempenho do hardware gráfico isolado. O escalonamento Reativo por Taxa (EMA) atinge um bom tempo (44,92 ms), mas sofre de miopia local em otimizações gulosas. Por fim, o **SA Adaptativo proposto** não apenas escapa de vales subótimos (graças à aceitação de Metropolis), como encontra o ponto dinâmico de balanceamento (ex.: 789 blocos para GPU e 235 para CPU), atingindo o recorde da plataforma de 35,89 ms. Este resultado comprova a premissa de que cooperar com a CPU mascara o gargalo do barramento e eleva a vazão global para cerca de **7,30 milhões de sistemas resolvidos por segundo**.

5.7. Projeção para as classes do NPB SP e estado da arte

Com as taxas medidas (3,75M sistemas/s pelo EMA; 4,86M/s no mega-lote com SA), o relatório projeta a *fase pentadiagonal* das classes do NPB SP (três varreduras $\times N^2$ linhas $\times$ passos): Classe C (162³, 400 passos) $\approx$ 4,2 s; Classe D (408³, 500 passos) $\approx$ 66,6 s; Classe E (1020³, 500 passos) $\approx$ 416,5 s (Figura 7(a)). A Figura 7(b) posiciona esses números frente aos tempos publicados do SP Classe C completo (benchmark inteiro, incluindo RHS e comunicação), de 29,3 s (Titan) a 483,24 s (serial). Embora a comparação seja estrita à fase pentadiagonal e não ao benchmark completo, ela evidencia que a fase dominante de uma malha com mais de 1 bilhão de pontos (Classe E) é executável em segundos em um único nó heterogêneo, escala que a especificação original do NPB projetou para execução distribuída em sistemas de mais de 1000 processadores em paralelo [Bailey et al. 1993].

5.8. Validação Transgeracional e Resiliência Out-of-Core

Para avaliar a portabilidade e a robustez do despachante adaptativo frente a variações severas de hardware, o solver com a malha de $1,061 \times 10^9$ pontos (4064 blocos, $n = 1020$) foi submetido a execução em uma plataforma restrita (CPU quad-core, 3 threads úteis e GPU NVIDIA GeForce GTX 660 com arquitetura Kepler sm_30 e 2 GB de VRAM) sob regime ativo de paginação em disco (swap em SSD).

O mecanismo de streaming em mega-lotes permitiu o processamento contínuo da malha sem estouro de memória física de vídeo (VRAM). Sob contenção de barramento e alta latência de disco no host — na qual o tempo médio de resposta da CPU degradou transitariamente para 138,9 ms/bloco —, o controlador SA detectou o afogamento em tempo real e redirecionou dinamicamente o fluxo para a GPU (1,38 ms/bloco), despachando 1586 blocos para o dispositivo e concluindo o ciclo completo em 90,87 s. O checksum final reproduziu exatamente o valor de referência ($1,889440 \times 10^8$), comprovando a bit-

identidade estrita entre gerações de silício (sm_30 vs. sm_89) e a tolerância a ruído sistêmico.

6. Discussão

Eficiência do ajuste dinâmico. A evidência empírica mais forte é que o modo automático superou os dois modos fixos na mesma máquina, validando a hipótese central: o custo real de cada fração é uma função não-monótona do estado transiente da máquina, inacessível a modelos estáticos — apenas a medição em tempo real com busca heurística (SA) captura o ótimo. O custo de adaptação é pequeno (uma janela de SA_WIN blocos, custo de medição negligenciável frente à varredura), e o modo congelado ($T < T_{freeze}$) garante estabilidade em execuções longas.

Eficiência de Recursos. O gargalo de memória de matrizes bandadas não exige escalonamento horizontal massivo. A combinação de lista livre, fatoração LU e ajuste por SA alcança vazões de trilhões de pontos em nós de trabalho únicos.

6.1. Resiliência Out-of-Core e Varreduras Tridimensionais

Na execução da Classe E (16,8 GB) num nó legado de 15 GiB e GTX 660 2 GB (swap 32 GiB SSD), a paginação gerou *thrashing*. O controlador EMA acumulou esse ruído (inflando a latência da CPU para 266,89 ms/bloco). O decisor SA filtrou os picos transitórios, contornando o afogamento (Tabela 6).

Tabela 6. Resiliência Out-of-Core (swap SSD). O EMA falha sob thrashing; o SA converge.

Eixo (Modo)	Tempo Varredura	Ruído CPU (Latência)	Split (α)	Vazão SSD
X (EMA Baseline)	94,12 s	266,89 ms/bloco	36% GPU / 63% CPU	—
X (SA Nativo)	90,87 s	138,97 ms/bloco	39% GPU / 61% CPU	186,6 MB/s
Y (SA Transpose)	91,98 s	estável	42% GPU / 58% CPU	184,4 MB/s
Z (SA Transpose)	212,43 s	afogamento I/O detectado	21% GPU / 79% CPU	79,8 MB/s

Para transpor os saltos descontínuos em Y e Z , as threads *host* linearizaram blocos em micro-buffers de 16 MB. Em Y , operou na taxa de X . Em Z (salto de 8 MB/ponto), saturou I/O para 79,8 MB/s; o SA detectou a contenção e deslocou 79% para a CPU, concluindo a malha 3D.

Adaptação Autônoma a Acessos Descontínuos (X, Y, Z). O comportamento divergente entre os eixos X/Y e o eixo Z evidencia a utilidade prática do Simulated Annealing sem modelo prévio (*zero-profiling*). Enquanto nos eixos X e Y o empacotamento do *Edge Transpose* manteve a GPU saturada ($\alpha \approx 39\text{--}42\%$), a dispersão extrema de memória no eixo Z induziu contenção na controladora do SSD. Em sistemas estáticos, essa desaceleração forçaria a GPU a aguardar transferências lentas por DMA; o controlador SA, contudo, capturou o atraso relativo e priorizou a computação nos núcleos de CPU locais, provando que a meta-heurística reage tanto a variações térmicas de processamento quanto a gargalos físicos de subsistemas de armazenamento e memória virtual.

Limitações e Validação. (i) A comparação de GFLOPS com o NPB-GPU usa contagens de flops distintas (28 vs. 850 flops/ponto) e o NPB-GPU compila com `-use_fast_math`, enquanto o presente trabalho emprega aritmética estrita (`-fmad=false`); por essa razão, métricas invariantes de vazão (pontos/s e sistemas lineares/s) são adotadas como base primária. (ii) A margem de 13% sobre o modo somente-GPU é dependente do equilíbrio térmico e do barramento da máquina avaliada; o ganho

arquitetural estrutural é a redução frente ao modo somente-CPU ($3,7\times$). (iii) O pipeline cobre agora as três direções espaciais (X , Y e Z) com *Edge Transpose*; a integração da avaliação do termo de fonte (RHS) ao passo temporal completo do NPB SP constitui a etapa subsequente.

7. Conclusão e trabalhos futuros

Este trabalho apresentou um solver pentadiagonal híbrido CPU–GPU com balanceamento adaptativo por simulated annealing, telemetria térmica e garantia de reprodutibilidade bit-a-bit. Na plataforma de referência (AMD EPYC + NVIDIA RTX 5000 Ada), o solver resolveu 1,061 trilhão de pontos em 213,9 s, atingindo vazão sustentada de 4,86 milhões de sistemas lineares por segundo e superando os modos puramente GPU (em 13%) e puramente CPU (em $3,7\times$). O kernel de produção superou o *cuPentBatch* em $1,05\times$ no solve e em $2,17\times$ no ciclo completo, mantendo resíduo $\|Ax - b\|_\infty < 10^{-15}$.

A validação em nó legado restrito confirmou a robustez *out-of-core* com transposição dinâmica (X , Y , Z), comprovando a independência de alocação monolítica. Sob gargalos de paginação e stride de 8 MB, o controlador SA adaptou o balanceamento autonomamente, democratizando malhas de 1 bilhão de pontos com bit-identidade em estações de entrada. Trabalhos futuros contemplam a expansão para multi-GPU e o passo temporal SP completo.

O código-fonte do solver, as campanhas de revalidação e o artigo estão disponíveis publicamente em [linkomitidopararevisoeaga].

Referências

- Araujo, G., Griebler, D., Rockenbach, D. A., Danelutto, M., and Fernandes, L. G. (2021). NAS parallel benchmarks with CUDA and beyond. *Software: Practice and Experience*, 51(11):2257–2280.
- Augonnet, C., Thibault, S., Namyst, R., and Wacrenier, P.-A. (2011). StarPU: a unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency and Computation: Practice and Experience*, 23(2):187–198.
- Bailey, D., Barton, J., Lasinski, T., and Simon, H. (1993). The NAS parallel benchmarks. NASA Technical Memorandum 103863, NASA Ames Research Center, Moffett Field, CA, USA.
- Bailey, D. H., Barszcz, E., Barton, J. T., Browning, D. S., Carter, R. L., Dagum, L., Fatoohi, R. A., Frederickson, P. O., Lasinski, T. A., Schreiber, R. S., Simon, H. D., Venkatakrisnan, V., and Weeratunga, S. K. (1991). The NAS parallel benchmarks. In *Proceedings of the ACM/IEEE Conference on Supercomputing (SC'91)*, pages 158–165.
- Carroll, E., Gloster, A., Ó Náraigh, L., and Pang, K. E. (2019). cuPentBatch — a batched pentadiagonal solver for NVIDIA gpus. *Computer Physics Communications*, 241:79–89.
- Chang, L.-W., Stratton, J. A., Kim, H.-S., and Hwu, W.-M. W. (2012). A scalable, numerically stable, high-performance tridiagonal solver using GPUs. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC'12)*. IEEE Computer Society.

- Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, USA, 4th edition.
- Jackson, C. W. and Zubair, M. (2021). Improvements to a batch pentadiagonal solver on NVIDIA gpus. Technical report, NASA Langley Research Center. NASA Technical Report 20210009602; pôster no SIAM CSE21, virtual, 1–5 de março de 2021.
- Kang, J.-H., Kim, K.-H., Pan, X., and Choi, J.-I. (2023). PaScaL-TDMA 2.0: A multi-GPU-based library for solving massive tridiagonal systems. *Computer Physics Communications*, 291:108820.
- Kim, S., Kim, J., Ha, S., and You, D. (2025). A highly scalable TDMA for GPUs and its application to flow solver optimization. *arXiv preprint arXiv:2509.03933*.
- Kirkpatrick, S., Gelatt, C. D., and Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598):671–680.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., and Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092.
- NVIDIA Corporation (2023). *NVIDIA Management Library (NVML) Reference Manual*. NVIDIA Corporation, Santa Clara, CA, USA.
- Thomas, L. H. (1949). Elliptic problems in linear difference equations over a network. Technical report, Watson Scientific Computing Laboratory, Columbia University, New York.
- Tolmachev, D., Marti, P., Castiglioni, G., and Jackson, A. (2025). High performance solution of tridiagonal systems on the GPU. *ACM Transactions on Parallel Computing*, 12(2):5:1–5:25.